OCR Validation Subsystem — Architecture

Component: Document OCR Validation Pipeline **Location:** `hxgn-sdx-bot/bot/` — Step 8b of the SDx automation flow **Owner:** Document Control Automation **Date:** 2026-04-15

---

1. C4 — Context

```
┌──────────────────┐      ┌──────────────────────┐          ┌──────────────────┐
│   Hexagon SDx    │      │                      │          │  LLM Provider    │
│     Portal       │─ PDF ──▶│   OCR Validation     │──prompt─▶│  Gemini / Azure  │
│   (source of     │◀─ verdict│     Subsystem        │◀─ JSON ──│     OpenAI       │
│   truth for      │      │                      │          │                  │
│   submittals)    │      └──────────────────────┘          └──────────────────┘
└──────────────────┘                 │
                                     │
                                     ▼
                            ┌──────────────────┐
                            │  Reviewer (DC)   │
                            │  reads annotated │
                            │  PDF + HTML rpt  │
                            └──────────────────┘
```

---

2. C4 — Containers

```
┌────────────────────────────────────────────────────────────────────────┐
│                        OCR Validation Subsystem                         │
│                                                                         │
│  ┌────────────────┐   ┌─────────────────┐   ┌────────────────────────┐  │
│  │  Orchestrator  │   │  OCR Engine     │   │  Annotation Service    │  │
│  │  step08b-*.js  │──▶│  (strategy:     │──▶│  generateHighlightedPDF() │  │
│  │                │   │  Gemini │ Azure) │   │  pdfjs-dist + pdf-lib  │  │
│  └───────┬────────┘   └────────┬────────┘   └───────────┬────────────┘  │
│          │                     │                        │               │
│          ▼                     ▼                        ▼               │
│  ┌────────────────┐   ┌─────────────────┐   ┌────────────────────────┐  │
│  │  Reporter      │   │  Rule Engine    │   │  Artefact Store        │  │
│  │  HTML + JSON   │   │  filterIssues() + │   │  downloads/ + reports/ │  │
│  │                │   │  loadsheet / CP2 │   │                        │  │
│  └────────────────┘   └─────────────────┘   └────────────────────────┘  │
└────────────────────────────────────────────────────────────────────────┘
```

---

3. C4 — Components (runtime view)

```
                 ┌──────────────────────────────────────────────┐
                 │             Orchestrator (Step 8b)           │
                 │  • loads .env (OCR_PROVIDER)                 │
                 │  • downloads PDF from SDx                    │
                 │  • selects engine via factory                │
                 │  • assembles final verdict                   │
                 └────────────────────┬─────────────────────────┘
                                      │ runOCRValidation(pdf, loadsheet, submittal)
                                      ▼
      ┌──────────────────────────────────────────────────────────────┐
      │                        OCR Engine                            │
      │  (interchangeable · identical contract)                      │
      │                                                              │
      │   ┌────────────────────────┐    ┌────────────────────────┐   │
      │   │  GeminiAdapter         │    │  AzureOpenAIAdapter    │   │
      │   │  vision-native (PDF)   │    │  text-first (pdf-parse)│   │
      │   │  gemini-2.5-flash      │    │  gpt-4o · JSON mode    │   │
      │   │  retry 20/35/50s       │    │  retry 20/35/50s       │   │
      │   └────────────────────────┘    └────────────────────────┘   │
      │                                                              │
      │   Pipeline (executed per document):                          │
      │                                                              │
      │   ① EXTRACT      refData = {doc#, rev, title, date,          │
      │                             project, contract, discipline…} │
      │   ② AUDIT        issues[], blank[], non_ocr[],               │
      │                  watermark, markups, signatures,             │
      │                  revision_history, security_class            │
      │   ③ RECONCILE    refData  ↔  loadsheet                        │
      │   ④ GATE         project ∈ CP2 ; Rev 0 ≠ AFC                  │
      └───────────────────────────────┬──────────────────────────────┘
                                      │ { checks[], refData, analysis, isValid }
                                      ▼
                 ┌──────────────────────────────────────────────┐
                 │        Annotation Service                    │
                 │  pdfjs-dist  → text bbox coordinates         │
                 │  pdf-lib     → yellow fill / red border /    │
                 │                whole-page stamp / legend     │
                 │  output: <doc>_errors.pdf                    │
                 └────────────────────┬─────────────────────────┘
                                      ▼
                 ┌──────────────────────────────────────────────┐
                 │        Reporter                              │
                 │  JSON  → reports/last-run-log.json           │
                 │  HTML  → reports/report-<ts>.html            │
```

```
          │  Verdict → SDx (ACCEPT / REJECT)               │
          └────────────────────────────────────────────────┘
```

---

4. Sequence — single document validation

```
Orchestrator         OCR Engine             LLM             Annotation       Reporter
     │                    │                    │                    │                │
     │ runOCRValidation() │                    │                    │                │
     ├───────────────────►│                    │                    │                │
     │                    │ ① extract prompt   │                    │                │
     │                    ├───────────────────►│                    │                │
     │                    │◄──── refData ──────┤                    │                │
     │                    │                    │                    │                │
     │                    │ ② audit prompt     │                    │                │
     │                    ├───────────────────►│                    │                │
     │                    │◄──── analysis ─────┤                    │                │
     │                    │                    │                    │                │
     │                    │ ③ reconcile loadsheet (in-process)      │                │
     │                    │ ④ CP2 gate        (in-process)          │                │
     │                    │                    │                    │                │
     │                    │ failedIssues[]     │                    │                │
     │                    ├────────────────────────────────────────►│                │
     │                    │                    │                    │ annotated PDF  │
     │                    │                    │                    ├───────────────►│
     │◄──────────── checks[] + isValid ─────────────────────────────┤                │
     │                                                                               │
     │ publish verdict + artefacts                                                   │
     ├──────────────────────────────────────────────────────────────────────────────►│
     │                                                                               │
```

---

5. Design decisions & rationale

#	Decision	Rationale
1	**Strategy pattern for LLM provider** (`OCR_PROVIDER=gemini|azure`)	Vendor independence; swap without touching the orchestrator. Identical public contract enforced by shared reducer.
2	**Two-phase prompting** (extract → audit)	Separates *ground truth capture* from *discrepancy hunting*; keeps each prompt small and deterministic (`temperature=0`, JSON mode).
3	**Deterministic override for non-OCR pages** (Azure)	`pdf-parse` per-page char count is more reliable than the LLM for blank/image-only detection — a safety net against hallucination.
4	**Post-LLM false-positive filter** (`filterIssues`)	Normalises whitespace / case and reconciles 11 date formats before surfacing a failure — prevents reviewer fatigue.
5	**Rule engine in code, not in the prompt** (CP2 gate, loadsheet reconciliation)	Business rules are auditable, unit-testable, and cheap; only the linguistic work is delegated to the LLM.
6	**Per-issue traceability** (`{page, field, found, expected}`)	Enables precise bbox highlighting downstream; makes every failure defensible to an auditor.
7	**Exponential back-off (20/35/50s) on 429**	Matches both providers' published rate-limit recovery windows; avoids cascading failure during batch runs.

---

6. Quality attributes

Attribute	How it is achieved
Reliability	Retry with back-off; deterministic overrides; structured JSON responses.
Traceability	Every finding carries page + field + found + expected; artefacts persisted per run.
Explainability	Annotated PDF renders failures in-context for human reviewers.
Portability	Provider-agnostic interface; env-var switch; no prompt duplication beyond provider I/O shape.
Cost control	Text-first path (Azure) avoids vision tokens; Gemini path used only when native PDF is preferred.
Security	API keys in `.env`; no PDF content logged; LLM calls use JSON-only response format.

---

7. Inputs / Outputs (contract)

Input

- `pdfPath` — absolute path to downloaded submittal PDF
- `loadsheet` — `{ documentNumber, title, revision, discipline }`
- `submittal` — SDx metadata snapshot

Output

- `checks[]` — ordered list of `{name, status: PASS|WARN|FAIL, note, page?, field?}`
- `refData` — canonical metadata extracted from the PDF
- `analysis` — structural flags + filtered issues
- `isValid` — boolean gate consumed by Step 9 (approve/reject)
- `highlightedPdfPath` — annotated artefact for the reviewer

---

8. File map

Role	Path
Orchestrator	`hxgn-sdx-bot/bot/steps/step08b-document-validation.js`
Gemini adapter	`hxgn-sdx-bot/bot/utils/gemini-ocr.js`
Azure OpenAI adapter	`hxgn-sdx-bot/bot/utils/azure-openai-ocr.js`
Annotation service	`generateHighlightedPDF()` in `gemini-ocr.js`
Reporter	`hxgn-sdx-bot/bot/utils/reporter.js`
Artefact store	`hxgn-sdx-bot/bot/downloads/` · `hxgn-sdx-bot/bot/reports/`
